

TRABAJO PRÁCTICO – SISTEMA JUDICIAL CON PERSISTENCIA POLÍGLOTA

Alumno

Santiago Viggiano

Carrera

Licenciatura en Sistemas

1. Introducción

En los sistemas modernos resulta cada vez más frecuente la necesidad de almacenar y procesar información con características muy diferentes. En estos escenarios, utilizar una única tecnología de base de datos suele generar limitaciones en términos de rendimiento, flexibilidad y capacidad de análisis.

Como respuesta a esta problemática surge el concepto de Persistencia Políglota (Polyglot Persistence), una estrategia que propone utilizar múltiples tecnologías de almacenamiento dentro de una misma solución, seleccionando en cada caso la herramienta más adecuada según el tipo de información y las operaciones requeridas.

El objetivo de este trabajo consistió en diseñar e implementar un Sistema Judicial basado en una arquitectura políglota, integrando tres motores NoSQL distintos:

- MongoDB para la gestión documental de expedientes y actuaciones.
- Neo4j para el modelado de relaciones entre personas y causas judiciales.
- Redis para la gestión operativa en tiempo real.

La aplicación fue desarrollada en Python y permite ejecutar diferentes operaciones judiciales utilizando de forma integrada los tres motores de persistencia.

2. Objetivos

Objetivo General

Desarrollar un sistema judicial utilizando una arquitectura de persistencia políglota que permita aprovechar las fortalezas específicas de MongoDB, Neo4j y Redis.

Objetivos Específicos

- Modelar expedientes judiciales mediante documentos NoSQL.
 - Representar relaciones complejas mediante grafos.
 - Gestionar información temporal y operativa utilizando estructuras en memoria.
 - Integrar múltiples tecnologías de persistencia dentro de una única aplicación.
 - Implementar operaciones que involucren simultáneamente más de un motor de base de datos.
-

3. Arquitectura de la Solución

La solución implementada se basa en tres motores especializados, cada uno destinado a resolver un conjunto particular de necesidades.

MongoDB

MongoDB fue utilizado como repositorio principal de información documental.

En el ámbito judicial, los expedientes contienen múltiples atributos y estructuras que pueden evolucionar con el tiempo. El modelo orientado a documentos de MongoDB permite almacenar esta información de manera flexible sin necesidad de definir esquemas rígidos.

Además, facilita la recuperación completa de expedientes y actuaciones mediante consultas simples y eficientes.

Neo4j

Neo4j fue incorporado para representar relaciones entre entidades judiciales.

Dentro de un sistema judicial resulta habitual la necesidad de identificar vínculos entre expedientes, abogados, imputados y demás participantes.

La utilización de grafos permite recorrer estas conexiones de manera eficiente, facilitando la detección de conflictos de interés y la identificación de causas relacionadas.

Redis

Redis fue utilizado para gestionar información operativa que requiere tiempos de respuesta muy bajos.

Entre sus principales usos dentro del sistema se encuentran:

- Administración de salas.
- Agenda de audiencias.
- Control de accesos temporales.
- Gestión de notificaciones.

Su naturaleza en memoria permite realizar estas operaciones prácticamente en tiempo real.

4. Modelo de Datos Implementado

4.1 MongoDB

Se crearon las siguientes colecciones:

Personas

Contiene información básica de los participantes del sistema.

Campos principales:

- persona_id
- nombre
- apellido
- rol

Cantidad cargada: 300 registros.

Expedientes

Representa las causas judiciales almacenadas en el sistema.

Campos principales:

- expediente_id
- carátula
- fuero
- juzgado
- estado
- imputados
- abogados

Cantidad cargada: 200 expedientes.

Actuaciones

Registra eventos y acciones realizadas sobre cada expediente.

Cantidad cargada: 500 actuaciones.

Audiencias

Almacena audiencias programadas.

Cantidad cargada: 100 audiencias.

Resoluciones

Contiene resoluciones judiciales emitidas.

Cantidad cargada: 80 resoluciones.

4.2 Neo4j

Se implementó un modelo basado en nodos y relaciones.

Nodos

Persona

Representa individuos que participan en procesos judiciales.

Cantidad: 300 nodos.

Expediente

Representa causas judiciales.

Cantidad: 200 nodos.

Relaciones

INTERVIENE_EN

Relaciona personas con expedientes.

Cantidad: 400 relaciones.

REPRESENTA

Relaciona abogados con personas representadas.

Cantidad: 100 relaciones.

CONEXO_A

Relaciona expedientes vinculados entre sí.

Cantidad: 50 relaciones iniciales más las generadas durante la ejecución del sistema.

Estas relaciones permiten detectar posibles conflictos de interés y analizar conexiones entre causas.

4.3 Redis

Se utilizaron diversas estructuras nativas de Redis.

Hashes

Representan el estado operativo de las salas judiciales.

Ejemplo:

sala:1

Atributos:

- estado
 - juzgado
 - expediente_activo
 - hora_fin_estimada
-

Sorted Sets

Utilizados para administrar la agenda diaria de audiencias.

Operaciones implementadas:

- Registro de audiencias.
 - Obtención de próxima audiencia.
 - Consulta de audiencias próximas.
 - Detección de solapamientos.
 - Cálculo de carga diaria.
-

Claves Temporales (TTL)

Utilizadas para el control de acceso a expedientes.

Cada permiso posee una duración limitada de dos horas.

Operaciones implementadas:

- Generación de acceso temporal.
 - Verificación de acceso vigente.
 - Revocación manual de acceso.
-

Streams

Utilizados para registrar eventos operativos del sistema.

Ejemplos:

- inicio_audiencia
- fin_audiencia

Estas estructuras permiten mantener un historial de eventos generados durante la operación del sistema.

5. Operaciones Implementadas

La aplicación incorpora un menú principal que permite ejecutar distintas operaciones judiciales.

OP-1: Panel Operativo del Juzgado

Motores utilizados:

- Redis

Funcionalidades:

- Consulta del estado de las salas.
 - Visualización de salas disponibles.
 - Consulta de agenda diaria.
 - Obtención de próxima audiencia.
 - Detección de solapamientos.
 - Consulta de carga diaria.
 - Visualización de notificaciones recientes.
-

OP-2: Apertura de Audiencia

Motores utilizados:

- Redis
- MongoDB
- Neo4j

Proceso:

1. Redis actualiza el estado de la sala.
 2. Redis registra un evento de inicio.
 3. MongoDB almacena la actuación correspondiente.
 4. Neo4j verifica la existencia de causas conexas.
-

OP-3: Búsqueda de Expediente y Control de Acceso

Motores utilizados:

- Redis
- MongoDB

Proceso:

1. Redis verifica la existencia de permisos temporales.
 2. Si el permiso no existe, se genera automáticamente.
 3. MongoDB recupera el expediente solicitado.
-

OP-4: Detección de Conflicto de Interés

Motores utilizados:

- Neo4j
- MongoDB

Proceso:

1. Neo4j analiza relaciones entre expedientes.
 2. Se identifican causas vinculadas.
 3. MongoDB registra una actuación de alerta.
-

OP-5: Cierre de Audiencia

Motores utilizados:

- Redis
- MongoDB
- Neo4j

Proceso:

1. Redis libera la sala utilizada.
 2. Redis registra un evento de cierre.
 3. MongoDB almacena el acta correspondiente.
 4. Neo4j genera nuevas relaciones derivadas de la audiencia.
-

6. Resultados Obtenidos

Durante las pruebas realizadas se verificó el correcto funcionamiento de los tres motores de persistencia y de todas las operaciones implementadas.

Se validó exitosamente:

- La carga automática de datos.
- La consulta de expedientes en MongoDB.
- La detección de relaciones mediante Neo4j.
- La administración de salas y accesos temporales mediante Redis.
- La integración de múltiples motores dentro de una misma operación.

Las funcionalidades implementadas demostraron la viabilidad de una arquitectura basada en persistencia polígloa para un dominio complejo como el judicial.

7. Conclusiones

La implementación desarrollada permitió comprobar las ventajas de utilizar diferentes tecnologías de persistencia dentro de una misma solución.

MongoDB proporcionó una estructura flexible y eficiente para la gestión documental de expedientes y actuaciones.

Neo4j permitió representar y analizar relaciones complejas entre entidades judiciales, facilitando la detección de conflictos de interés y causas relacionadas.

Redis ofreció un mecanismo altamente eficiente para la gestión operativa en tiempo real, incluyendo salas, agendas, accesos temporales y notificaciones.

Como resultado, se obtuvo una solución escalable, flexible y alineada con los principios de persistencia políglota, aprovechando las fortalezas particulares de cada motor de base de datos.

Decisión Tecnológica: Utilización de Python

Inicialmente se evaluó la posibilidad de desarrollar la aplicación utilizando Java, debido a su amplia utilización en sistemas empresariales y su robustez para el desarrollo de aplicaciones de gran escala.

Sin embargo, considerando el alcance del trabajo práctico y los objetivos principales de la actividad, se decidió implementar la solución utilizando Python.

Esta decisión estuvo fundamentada en diversos aspectos técnicos:

- Disponibilidad de bibliotecas oficiales para MongoDB, Neo4j y Redis.
- Menor complejidad de configuración e integración entre tecnologías.
- Reducción significativa de la cantidad de código necesario para implementar las operaciones requeridas.
- Mayor velocidad de desarrollo y prueba de funcionalidades.
- Facilidad para realizar cargas masivas de datos y consultas exploratorias.

Dado que el objetivo principal del trabajo consistía en demostrar la aplicación de conceptos de persistencia políglota y el uso integrado de múltiples motores NoSQL, Python permitió concentrar el esfuerzo en el diseño de la arquitectura y en la implementación de las operaciones de negocio, evitando complejidades adicionales asociadas al desarrollo de infraestructura y configuración propias de otros entornos.

Por estos motivos, Python fue seleccionado como lenguaje de implementación principal, permitiendo desarrollar una solución funcional, modular y fácilmente extensible.